Frontend Recreation Plan

AI-Orchestration Content Factory

Component	Tech Stack	Lines of Code
Frontend (apps/web)	React 19, TypeScript 5.9, Tailwind CSS v4, Vite 8, SWR, DnD Kit	~3,830 (68 files)
API Layer (apps/api)	FastAPI, SSE, Supabase Realtime, httpx	~2,500 (16 files)
Backend Packages	Python, LangGraph, Supabase, FreeRouter v3.1 (LiteLLM proxy)	~35,000+ (Python)
FreeRouter	Standalone LLM proxy (LiteLLM), 2 providers, 7 routes	Independent

Prepared for: Hassan Arif

Repository: github.com/HassanArif-collab/AI-Orchestration

April 2026

Branch: codebase-audit-finding-fixes

Table of Contents

1. Executive Summary

This document provides a comprehensive analysis of the frontend codebase of the AI-Orchestration project (Content Factory), specifically targeting the codebase-audit-finding-fixes branch. The analysis covers all 68 frontend source files across the React application (apps/web), identifying 17 critical bugs, 23 high-priority issues, and 31 medium-priority improvements. Based on this thorough audit, the document presents a detailed 10-chunk recreation plan designed to rebuild the frontend from scratch in independently testable, incrementally deliverable units.

The current frontend is a Kanban-based dashboard for managing an AI-powered YouTube content pipeline. It integrates with a FastAPI backend via REST endpoints, SSE streaming for chat, and Supabase Realtime WebSockets for live card updates. While the underlying concept is solid, the AI-generated implementation suffers from pervasive architectural issues: uncontrolled re-rendering in streaming components, fragile focus management in modals, missing error boundaries, and inconsistent API contracts. These are not isolated bugs but systemic patterns that resist piecemeal patching.

The recommended approach is a selective recreation: rebuild apps/web and apps/api from the ground up using modern patterns, while surgically fixing the identified bugs in the packages layer (which contains ~35,000 lines of complex, mostly correct business logic). Each chunk in the plan is designed to be independently verifiable against the existing backend, ensuring continuous validation throughout the rebuild process.

Metric	Current State	Target State
Frontend Files	68 files, ~3,830 LOC	Clean rebuild, ~2,500 LOC
Critical Bugs	17 identified	0 at each chunk gate
High-Priority Issues	23 identified	0 at each chunk gate
Recreation Chunks	N/A	10 independent chunks
API Endpoints Consumed	16 (inconsistent contracts)	16 (strict TypeScript types)
SSE/WebSocket Streams	2 (fragile parsing)	2 (robust with reconnection)

Table 1. Frontend Recreation Key Metrics

2. Current Architecture Analysis

2.1 Technology Stack

The frontend is built on a modern but bleeding-edge stack. React 19.2 and Vite 8 are both very recent releases, and while they provide excellent developer experience and performance characteristics, the ecosystem tooling has not fully caught up. Tailwind CSS v4 uses a CSS-first configuration approach (no tailwind.config.js), which is correct but means that all team members must be familiar with the new paradigm. The application uses SWR for data fetching, @dnd-kit for drag-and-drop on the Kanban board, and react-markdown with remark-gfm for rendering markdown content. Supabase provides both the PostgreSQL database and the Realtime WebSocket layer for live updates.

Technology	Version	Purpose	Risk Level
React	19.2.4	UI framework	High (very new)

Technology	Version	Purpose	Risk Level
Vite	8.0.1	Build tool	High (very new)
TypeScript	5.9.3	Type safety	Low (stable)
Tailwind CSS	4.2.2	Styling (CSS-first config)	Medium (v4 paradigm)
SWR	2.4.1	Data fetching + caching	Low (stable)
@dnd-kit/core	6.3.1	Kanban drag-and-drop	Low (stable)
Supabase JS	2.100.1	DB + Realtime WebSocket	Low (stable)
react-markdown	10.1.0	Markdown rendering	Low (stable)

Table 2. Frontend Technology Stack

2.2 Component Hierarchy

The application is organized as a single-page layout with no client-side routing. The component tree has a maximum depth of 4 levels, which is reasonable. The layout consists of a Header (topic discovery trigger), a central Kanban Board (6 columns with draggable cards), and a right-side Sidebar with 6 tabs (Chat, YouTube, Quota, Models, Skills, Knowledge Base). A CardDrawer overlays the board when a card is clicked, showing detailed information, agent activity logs, script previews, and the review/approval interface.

The data flow architecture uses two parallel patterns: Supabase Realtime WebSocket subscriptions for live Kanban card updates (managed through SWR), and traditional REST API calls for pipeline operations, quota monitoring, and chat streaming. The SSE streaming for chat uses raw fetch() + ReadableStream rather than the EventSource API, which is the correct approach since EventSource only supports GET requests and the chat endpoint requires POST with a body. Agent thoughts are received through a separate Supabase Realtime subscription on the agent_thoughts table.

Level	Components	Count
0 (Root)	MainLayout	1
1 (Layout)	Header, Board, Sidebar, ToastContainer	4
2 (Sections)	Column, Card, CardDrawer, ChatPanel, YouTubePanel, QuotaPanel, ModelRegistry, SkillViewer, KnowledgeBase	9
3 (Widgets)	StatusBadge, EmptyState, ProgressBar, AgentLog, ChatMessage, ScriptViewer, ReviewPanel, PublishConfirmModal, CompetitorCard, OwnStats	10
4 (Leaf)	ReactMarkdown (in multiple parents)	1

Table 3. Component Hierarchy Depth Map

2.3 Data Flow Architecture

The application employs a dual-channel data architecture that creates both flexibility and complexity. The primary data channel for Kanban cards and agent thoughts flows through Supabase Realtime (WebSocket) with SWR as the caching and revalidation layer. When a card is created, updated, or deleted in the backend, the Supabase postgres_changes trigger fires, the WebSocket delivers the event to the frontend, and SWR automatically revalidates the relevant cache key. This means the UI updates in near-real-time without explicit polling for card

operations.

The secondary data channel handles pipeline state, quota monitoring, and chat streaming. Pipeline state for a specific card is polled via SWR every 5 seconds (usePipelineState), while provider quotas are similarly polled every 5 seconds (useQuota). The chat system uses Server-Sent Events (SSE) over a POST request to /api/chat/stream, with the response parsed as a ReadableStream. This dual-channel approach is architecturally sound but introduces synchronization challenges: for example, a card moving from "review" status back to "drafting" after rejection is reflected via Supabase Realtime, but the pipeline state poller may still show stale state for up to 5 seconds.

Data Source	Transport	Hook/Module	Cache Key
Kanban Cards	Supabase Realtime + SWR	useCards	kanban-cards
Agent Thoughts	Supabase Realtime	useAgentStream	N/A (append-only)
Pipeline State	REST + SWR (5s poll)	usePipelineState	pipeline-state-{id}
Provider Quota	REST + SWR (5s poll)	useQuota	provider-quota
Chat Stream	SSE (POST + ReadableStream)	useChat	N/A (streaming)
YouTube Analytics	REST + SWR (5min poll)	useYouTube	competitor-videos

Table 4. Data Flow Architecture

3. Bug Catalog

The following tables present a comprehensive catalog of all identified bugs and issues in the frontend codebase, organized by severity. Each issue includes the affected file, a description of the problem, and its impact on the user experience or system stability. The severity ratings follow a standard classification: Critical bugs cause crashes or data loss; High-priority issues cause significant UX degradation; Medium-priority issues are design concerns that should be addressed during recreation; Low-priority items are minor improvements.

3.1 Critical Bugs (Must Fix)

Critical bugs represent failures that directly impact application stability, data integrity, or security. These issues would cause crashes, expose user data, or silently corrupt the application state if left unresolved. Each of these must be definitively addressed in the recreation, not merely patched.

#	File	Description	Impact
C1	Board.tsx	DragOverlay uses non-null assertion (cards.find()!) that crashes if card is deleted between drag start and overlay render	Runtime crash during DnD
C2	Sidebar.tsx	ChatPanel unmounts on tab switch, destroying SSE connection, message history, and session state	Chat data loss on navigation
C3	ChatPanel.tsx	Every streaming token re-renders entire message list, re-parsing markdown for all previous messages	Severe performance degradation
C4	ChatMessage.tsx	No rehype-sanitize on ReactMarkdown output from API responses creates XSS attack surface	Security vulnerability (XSS)

#	File	Description	Impact
C5	.env.example	Contains real Supabase project URL and anon key instead of placeholder values	Credential exposure
C6	api.ts	response.json() has no try/catch; invalid JSON throws SyntaxError that bypasses error mapper	Unhandled crash on malformed API response
C7	useChat.ts	No mutex on sendMessage; concurrent calls create competing AbortControllers with shared refs	Race condition in chat streaming

Table 5. Critical Bugs

3.2 High-Priority Issues

High-priority issues cause significant UX degradation or represent architectural problems that will compound if not addressed. While they may not cause immediate crashes, they create fragile patterns that make the codebase difficult to maintain and extend. These should all be resolved during the recreation process through better architectural decisions rather than individual patches.

#	File	Description	Impact
H1	ReviewPanel.tsx	No cancel/abort mechanism for hanging API calls; both Approve and Reject disabled during submit	User stuck on broken submit
H2	CardDrawer.tsx	Focus trap uses querySelector("button[onClick]") which is unreliable in React synthetic events	Broken focus management
H3	Card.tsx	handleSave and handleDelete do not call mutate() after success; card list does not refresh	Stale UI after actions
H4	Column.tsx	Badge shows active (non-expired) count but the list still renders expired cards, confusing UX	Misleading count display
H5	Sidebar.tsx	No resize listener; isOpen initial state uses window.innerWidth once on mount	Broken responsive behavior
H6	CardDrawer.tsx	Drawer closes immediately after onDecision callback; user cannot see result of approve/reject	Confusing UX flow
H7	PublishConfirmModal.tsx	No double-click prevention on publish button after countdown reaches zero	Accidental double-publish
H8	vite.config.ts	No API proxy configuration; causes CORS issues during development	Dev environment broken
H9	MainLayout.tsx	No error boundary wrapping; any render error crashes entire application	Full app white-screen
H10	api.ts	/api/kanban/cards/ vs /api/kanban/tasks/ - inconsistent resource naming in endpoint paths	API contract confusion
H11	useAgentStream.ts	Unhandled loadHistory rejection; subscribe() never fires if history fetch fails	Missing real-time updates
H12	useChat.ts	No SSE reconnection for dropped connections; only handles timeouts, not transient network failures	Permanent chat failure
H13	ScriptViewer.tsx	Side-by-side grid (lg:grid-cols-2) never activates inside 600px-wide drawer (lg breakpoint is 1024px)	Layout never triggers

#	File	Description	Impact
H14	CompetitorCard.tsx	No loading="lazy" on thumbnails, no onError handler, no aspect-ratio constraint	Broken image display

Table 6. High-Priority Issues

3.3 Medium-Priority Issues

Medium-priority issues represent design concerns, performance optimizations, and code quality improvements that should be addressed during recreation. While they do not cause immediate failures, they create technical debt that impacts maintainability, developer experience, and long-term application health. Many of these can be resolved naturally by adopting better patterns in the new implementation.

#	File	Description
M1	Board.tsx	grouped cards computed every render without useMemo; recalculates on unrelated state changes
M2	Card.tsx	useCardTimer runs interval for every card even those not in column 2 (expires_at is null)
M3	cardHelpers.ts	Column 1 has no action logic for error state; errored discovery cards get stuck with no retry option
M4	constants.ts	EXPIRATION_MS hardcoded at 3 hours with no sync mechanism to backend cron (which is 10 min cleanup)
M5	usePipelineState.ts	Polls every 5s even when pipeline is idle or in terminal state; wastes backend requests
M6	useQuota.ts	Polls globally regardless of quota panel visibility; wastes requests when panel is hidden
M7	useToast.ts	No maximum toast limit; rapid errors cause unlimited toast accumulation
M8	index.css	Duplicate animate-spin definition conflicts with Tailwind built-in animate-spin
M9	types/index.ts	MODEL_REGISTRY is hardcoded display data that drifts from actual backend model assignments
M10	errorMapper.ts	Missing 409 Conflict and 422 Unprocessable Entity handling for common API scenarios
M11	supabase.ts	No runtime type validation on Supabase data; schema drift silently produces wrong data
M12	useYouTube.ts	Dead code: apiFetcher function defined but never used
M13	KnowledgeBase.tsx	Same XSS concern as ChatMessage - renders Markdown from API without rehype-sanitize
M14	YouTubePanel.tsx	Competitor videos always fetched even when "Own Channel" tab is active
M15	multiple	Retry counts, timeouts, backoff delays scattered across 4 files with no centralized config

Table 7. Medium-Priority Issues

4. Chunk-by-Chunk Recreation Plan

The following plan divides the frontend recreation into 10 independently verifiable chunks. Each chunk is designed to produce a working, testable increment that can be validated against the existing backend before proceeding. The chunks are ordered by dependency: foundational layers (types, API client, layout) come first, followed by the core Kanban board, then detail views, and finally supplementary features. Each chunk specifies the files to create, the acceptance criteria for completion, the bugs it resolves, and the estimated scope.

Chunk	Name	Depends On	Estimated Files	Priority
0	Project Foundation	None	12	Critical
1	Types and API Client	Chunk 0	8	Critical
2	Layout Shell	Chunk 0, 1	6	High
3	Kanban Board Core	Chunk 1, 2	10	Critical
4	Card Details and Drawer	Chunk 3	8	High
5	Review and Approval Flow	Chunk 4	5	Critical
6	Chat Panel	Chunk 1, 2	6	High
7	Sidebar Panels	Chunk 2, 6	12	Medium
8	Telemetry and System	Chunk 1, 2	6	Medium
9	Polish and Responsive	All chunks	Cross-cutting	Medium

Table 8. Chunk Dependency Map

4.1 Chunk 0: Project Foundation

Goal: Set up the project scaffolding, build configuration, and development infrastructure that all subsequent chunks depend on. This chunk establishes the Vite + React + TypeScript + Tailwind CSS v4 project with proper path aliases, API proxy, environment configuration, error boundaries, and a global toast system. No visible UI is produced in this chunk, but the foundation must be rock-solid before any components are built.

Key Decisions: Use React 18 (stable) instead of React 19 to avoid ecosystem compatibility issues. Add a Vite proxy for /api requests to eliminate CORS during development. Implement path aliases (@/ for src/) to improve import readability. Set up a centralized constants/config module that replaces the scattered hardcoded values across the current codebase. Add rehype-sanitize as a mandatory dependency for all markdown rendering.

CRITICAL DEPENDENCY SPECIFICATIONS (April 2026 audit): The package.json MUST use these exact versions:

- "@supabase/supabase-js": "^2.101.0" - Critical security/performance fix (was ^2.45.0, 56 versions behind)
- "@dnd-kit/react": "^1.0.0" - Package restructured in late 2025. Replaces BOTH @dnd-kit/core AND @dnd-kit/sortable
- "tailwindcss": "^4.1.0" - v4.1.15 has container queries, 3D transforms, faster incremental builds
- "typescript": "^5.7.0" - Minor update from ^5.6.0
- "zod": "^3.23.0" - Runtime type validation for API responses
- "rehype-sanitize": "^6.0.0" - XSS prevention for all markdown rendering
- "zustand": "^5.0.0" - Global UI state management (sidebar, drawer, selected card, filters)
- "date-fns": "^3.6.0" - Timestamp formatting (useCardTimer hook)
- "lucide-react": "^0.460.0" - Icon library (replace emoji usage throughout)
- "clsx": "^2.1.0" - Conditional classname utility (cn() helper for Tailwind)
- "@radix-ui/react-dialog": "^1.1.0" - Accessible drawer/modal primitives
- "@radix-ui/react-tabs": "^1.1.0" - Accessible sidebar tab primitives (ARIA tablist pattern)
- "react-error-boundary": "^4.0.0" - Error boundary with retry UI and error reporting

- "@tanstack/react-virtual": "^3.10.0" - Virtual scrolling for columns with 30+ cards
- "swr": "^2.3.0" - Data fetching and caching (unchanged from current)

Do NOT install these deprecated/removed packages:

- "@dnd-kit/core" (replaced by @dnd-kit/react)
- "@dnd-kit/sortable" (merged into @dnd-kit/react)
- "framer-motion" (not needed - use CSS transitions and Tailwind animations)

STATE MANAGEMENT:

Use Zustand (^5.0.0) for global UI state. Create a single store at src/lib/store.ts with: sidebarOpen (boolean), activeTab (number - sidebar tab index), selectedCardId (string | null), drawerOpen (boolean), filters ({ column: number | null, search: string }). SWR remains the data-fetching layer (cards, pipeline state, quota). Zustand is only for UI state that does NOT come from API calls. Do NOT use React Context for this - Zustand's selector-based reactivity prevents unnecessary re-renders across components.

File	Purpose	Key Changes from Current
package.json	Dependencies and scripts	Use exact versions listed above. Do NOT use old versions.
vite.config.ts	Vite build configuration	Add /api proxy to localhost:3000, path aliases (@/), env prefix validation.
tsconfig.json	TypeScript configuration	Copy exactly with strict mode, ES2022 target, bundler module resolution, path aliases.
src/lib/store.ts	Global UI state (Zustand)	NEW: Zustand store for sidebarOpen, activeTab, selectedCardId, drawerOpen, filters.
.env.example	Environment template	REPLACE real credentials with placeholders. FreeRouter API keys in separate freerouter/.env. API_AUTH_ENABLED defaults to TRUE - document this.
.gitignore	Git ignore rules	Add .env to prevent accidental credential commits
src/index.css	Global styles	Remove duplicate animate-spin, keep Tailwind v4 config
src/main.tsx	App entry point	Wrap in ErrorBoundary, use extensionless imports
src/App.tsx	Root component	Add ErrorBoundary wrapper at root level
src/lib/config.ts	Centralized constants	NEW: Consolidate all scattered constants into one module
src/components/ErrorBoundary.tsx	Global error boundary	Use react-error-boundary (^4.0.0). Show error message, Reload button, Report button.
src/hooks/useToast.ts	Toast notification store	NEW: Add max toast limit (5), use crypto.randomUUID()
src/components/ui/ToastContainer.tsx	Toast UI component	Keep existing, add max limit enforcement

Table 9. Chunk 0 Files

Acceptance Criteria:

- npm run dev starts without errors and serves the app at localhost:5173
- API proxy correctly forwards /api/* requests to localhost:3000 without CORS issues
- Path alias @/ resolves correctly (import { X } from "@/lib/config")
- .env.example contains only placeholder values, no real credentials

- FreeRouter API keys are referenced via separate freerouter/.env, not in main .env
- ErrorBoundary catches render errors and shows a recovery UI instead of white-screen
- Toast system supports max 5 concurrent toasts with oldest-first dismissal
- TypeScript compiles in strict mode with zero errors (tsc --noEmit)
- npm run build produces a production bundle without warnings

Bugs Resolved: C5 (credential exposure), C6 (API proxy + JSON handling), H8 (CORS proxy), H9 (error boundary)

4.2 Chunk 1: Types and API Client

Goal: Establish the complete type system and API client layer that all components will depend on. This chunk creates a comprehensive, Zod-validated type system that matches the backend API contracts exactly, and a robust HTTP client with proper error handling, retry logic, and timeout configuration. The API client centralizes all endpoint definitions so that any contract mismatch is caught at compile time rather than at runtime.

Key Decisions: Use Zod for runtime type validation of API responses, not just TypeScript compile-time checks. This catches backend schema drift that TypeScript alone cannot detect (since API responses are typed as "any" at the fetch boundary). The API client should use an iterative retry approach instead of the current recursive pattern. All timeout and retry values should be configurable through the centralized config module. The chat SSE client should be a separate module with its own reconnection logic, not a bypass of the main API client.

File	Purpose	Key Changes from Current
src/types/index.ts	Domain types and display config	Add Zod schemas, constrain column type to 1-6 union, add CompetitorVideo type. CRITICAL: KanbanCard must include parent_id field (string | null). Also: id, title, description, column_index (NOT 'column'), topic_brief (jsonb), viability_score (numeric), expires_at (timestamptz | null), created_at, updated_at.
src/types/api.ts	API request/response schemas	NEW: Zod schemas for all 16 API endpoints
src/lib/api.ts	HTTP client with retry	Iterative retry (not recursive), try/catch on JSON parse, configurable timeouts per endpoint
src/lib/sse.ts	SSE streaming client	NEW: Reusable SSE client with reconnection, backoff, multi-line data support
src/lib/supabase.ts	Supabase client	Keep existing null-safety pattern, add connection health check
src/lib/errorMapper.ts	Error mapping	Add 409/422 handling, return original error for debugging
src/lib/cardHelpers.ts	Card action logic	Add Column 1 error handling, use proper discriminated union for actions
src/lib/constants.ts	App constants	Import from centralized config.ts, update MODEL_REGISTRY to remove Groq references

Table 10. Chunk 1 Files

Acceptance Criteria:

- All Zod schemas validate correctly against sample API responses from the backend
- API client handles invalid JSON gracefully with error mapper integration
- SSE client successfully connects to /api/chat/stream, parses events, and reconnects on disconnect

- SSE client handles multi-line data fields per SSE specification
- Error mapper produces actionable messages for 400, 401, 403, 404, 408, 409, 422, 429, 5xx status codes
- All API endpoint types are strictly typed; no "any" types in the public API surface
- cardHelpers returns correct action for all 6 columns including Column 1 error state
- MODEL_REGISTRY uses current FreeRouter provider names (OpenRouter, Ollama Cloud), not legacy Groq references

Bugs Resolved: C6 (JSON parse), M3 (Column 1 action), M9 (MODEL_REGISTRY drift), M10 (409/422), M15 (scattered config), C7 (SSE mutex via dedicated client)

4.3 Chunk 2: Layout Shell

Goal: Build the application layout skeleton that provides the visual structure for all subsequent chunks. This includes the Header with the topic discovery input, the MainLayout with proper flex arrangement, and the Sidebar with tab navigation. The critical requirement is that the Sidebar must NOT unmount tab content when switching tabs (using CSS display:none instead of conditional rendering) to preserve chat state and SSE connections.

Key Decisions: Use CSS-based tab visibility (display:none/block) instead of React conditional rendering for sidebar tabs. This prevents the critical bug where ChatPanel loses its SSE connection on tab switch. Add a resize listener to the Sidebar that responds to window width changes. Add a global loading state and empty state that can be shown before the Kanban board is mounted. The Header should debounce the discover action to prevent double-submission.

File	Purpose	Key Changes from Current
src/layout/Header.tsx	Top bar with discover input	Add aria-labels, debounce discover action, add input length validation
src/layout/MainLayout.tsx	Root layout structure	Add document.title management, ensure Board is always mounted
src/layout/Sidebar.tsx	Tabbed right panel	CRITICAL: CSS-based tab visibility (not conditional rendering), add resize listener
src/components/common/EmptyState.tsx	Empty state placeholder	Keep existing, no changes needed
src/components/common/StatusBadge.tsx	Pipeline status indicator	Ensure all 11 PipelineStatus values are handled (union of DiscoveryState + ProductionState). Do NOT add idle/thinking/waiting.
src/components/common/ProgressBar.tsx	Progress bar widget	Fix dynamic Tailwind class issue. IMPORTANT: No "progress" SSE event exists. Subscribe to iteration_complete, provider_status, or pipeline_update events.

Table 11. Chunk 2 Files

Acceptance Criteria:

- Switching sidebar tabs does NOT unmount ChatPanel (verify with React DevTools)
- Sidebar responds to window resize events (collapses on narrow viewports, expands on wide)
- All form inputs have proper aria-labels for screen reader accessibility
- Topic discovery input validates length and debounces submission
- Layout renders correctly on viewport widths from 1024px to 2560px
- Sidebar tabs follow proper ARIA tablist pattern with arrow key navigation

Bugs Resolved: C2 (ChatPanel unmount), H5 (resize listener), H9 (error boundary from Chunk 0)

4.4 Chunk 3: Kanban Board Core

Goal: Implement the central Kanban board with 6 columns, drag-and-drop card movement, optimistic updates, and real-time synchronization via Supabase Realtime. This is the most complex chunk and the heart of the application. Every card movement must be validated against the transition rules, optimistically applied to the UI, confirmed by the API call, and rolled back on failure. The board must handle the full lifecycle of a card from discovery through to publishing.

Key Decisions: Use useMemo for the grouped cards computation to prevent recalculation on unrelated state changes. Implement proper cleanup in the DnD handlers to prevent race conditions during rapid drags. Add safe fallbacks for the DragOverlay to prevent crashes when cards are deleted mid-drag. Use React.memo on Card components to prevent unnecessary re-renders when unrelated state changes. The Column component must filter expired cards consistently in both the badge count and the rendered list.

PERFORMANCE MANDATES:

1. useCards.ts: Do NOT call mutate() to refetch ALL cards on every Realtime event. Parse the Realtime INSERT/UPDATE/DELETE payload and apply directly to SWR cache using mutate(data, false). Only full refetch on subscribe() and reconnect.

2. Board.tsx: If a column has more than 30 cards, implement virtual scrolling using @tanstack/react-virtual (^3.x).

3. useCards.ts: The Supabase Realtime channel MUST call channel.unsubscribe() in the useEffect cleanup return.

4. DnD: Import from "@dnd-kit/react" (NOT "@dnd-kit/core"). The new API uses DragDropManager, useSortable, and useDraggable.

File	Purpose	Key Changes from Current
src/hooks/useCards.ts	Card data fetching + Realtime	Add optimistic update helpers, keep Realtime subscription pattern
src/hooks/useCardTimer.ts	Card expiration countdown	Only activate for cards in column 2, wrap parseISO in try/catch
src/components/kanban/Board.tsx	Central Kanban board	useMemo for grouped cards, safe DragOverlay fallback, fix race conditions
src/components/kanban/Column.tsx	Droppable column container	Fix expired card filtering (consistent badge + list), add drop highlight
src/components/kanban/Card.tsx	Draggable card component	Add React.memo, fix mutate() calls after save/delete, add delete confirmation
src/components/kanban/BoardSkeleton.tsx	Loading skeleton	NEW: Shimmer loading state for board initialization

Table 12. Chunk 3 Files

Acceptance Criteria:

- Cards can be dragged between valid columns and snap back on invalid drops
- Optimistic UI updates are immediately visible and properly rolled back on API failure
- Deleting a card during a drag operation does not crash the app

- Expired cards in Column 2 are filtered consistently in both badge count and card list
- useCardTimer only creates intervals for cards that actually have an expires_at value
- Board renders within 16ms (60fps) with up to 50 cards loaded
- Realtime updates from Supabase correctly reflect card movements from other sessions

Bugs Resolved: C1 (DragOverlay crash), H3 (missing mutate), H4 (badge mismatch), M1 (useMemo), M2 (timer optimization)

4.5 Chunk 4: Card Details and Drawer

Goal: Build the CardDrawer overlay that shows detailed information when a card is clicked. This includes the card metadata display, the agent activity log with real-time streaming via Supabase, the script preview panel, and the card action buttons. The drawer must properly manage focus, support keyboard navigation (Escape to close), and maintain its state when the review panel is opened.

Key Decisions: Replace the fragile querySelector-based focus trap with a proper library (like focus-trap-react) or a custom implementation that tracks focusable elements correctly in React. The drawer should NOT close automatically after approve/reject actions; instead, it should show the result of the action and let the user close it manually. Add a loading skeleton for agent thoughts while history is being fetched. The WebSocket connection error indicator should be based on connection state, not on string matching of error messages.

File	Purpose	Key Changes from Current
src/hooks/useAgentStream.ts	Real-time agent thoughts	Add .catch() on loadHistory chain, add connection state indicator
src/components/kanban/CardDrawer.tsx	Slide-out detail panel	Replace focus trap, add loading skeleton, do NOT close on decision
src/components/common/AgentLog.tsx	Agent thought entry	Ensure custom agent color classes are defined in Tailwind config
src/components/review/ScriptViewer.tsx	Script + visual cues viewer	Fix breakpoint for side-by-side (use md: instead of lg: inside drawer)

Table 13. Chunk 4 Files

Acceptance Criteria:

- Drawer opens on card click and closes on Escape key or backdrop click
- Focus is properly trapped inside the drawer using a React-compatible focus management approach
- Agent thoughts show a loading skeleton while history is being fetched from Supabase
- Failed history load does not prevent the realtime subscription from working
- Drawer remains open after approve/reject actions, showing the result of the action
- ScriptViewer side-by-side layout activates at the correct breakpoint for the drawer width

Bugs Resolved: H2 (focus trap), H6 (drawer closing), H11 (loadHistory), H13 (ScriptViewer breakpoint)

4.6 Chunk 5: Review and Approval Flow

Goal: Implement the human-in-the-loop review and approval workflow. This is a critical business flow where users review generated scripts and decide whether to approve for publishing or reject with feedback. The flow

includes the ReviewPanel with approve/reject buttons, a feedback textarea for rejections, and a safety countdown modal for the publish confirmation. This chunk also implements the pipeline state polling that detects when a card is waiting for review.

Key Decisions: Add an abort mechanism to the approve/reject API calls so users can cancel a hanging request. The ReviewPanel should use useReducer for the multi-step state machine (idle, confirming, submitting, success, error) instead of multiple useState calls. The publish confirmation modal should use a proper countdown component that prevents double-clicks. Pipeline state polling should stop or slow down when the card is in a terminal state.

File	Purpose	Key Changes from Current
src/hooks/usePipelineState.ts	Pipeline state polling	Conditional polling based on pipeline status, add stop-on-terminal-state
src/components/review/ReviewPanel.tsx	Approve/Reject UI	Add cancel/abort for API calls, use useReducer, add feedback max length
src/components/review/PublishConfirmModal.tsx	5s countdown publish modal	Fix focus trap, add double-click prevention, proper countdown cleanup

Table 14. Chunk 5 Files

Acceptance Criteria:

- Review panel appears when pipeline state indicates isWaitingForReview is true
- Approve flow: click Approve, optional publish modal with 5s countdown, confirm, card moves to Column 6
- Reject flow: click Reject, enter feedback (min 10 chars), submit, card returns to drafting with feedback
- Hanging API calls can be cancelled by the user (abort button visible during submit)
- Pipeline state polling stops when card reaches terminal state (complete, published)
- Publish button is disabled during countdown and re-enabled only after countdown completes

Bugs Resolved: H1 (cancel mechanism), H7 (double-click prevention), M5 (conditional polling), H13 (ScriptViewer from Chunk 4)

4.7 Chunk 6: Chat Panel

Goal: Build the chat interface with SSE streaming, tool usage tracking, suggestion chips, and a robust streaming experience. This is the second most complex chunk due to the real-time streaming requirements. The chat panel must render streaming tokens efficiently without re-rendering the entire message list on every token, support session persistence, and handle connection failures gracefully with automatic reconnection.

Key Decisions: Use React.memo on ChatMessage components to prevent markdown re-parsing on every token. Debounce the auto-scroll trigger so it does not fire on every single token (debounce to 100ms). Implement a streaming buffer that accumulates tokens and flushes to the UI at a fixed interval (e.g., every 50ms) rather than on every token. Add a proper reconnection mechanism to the SSE client (from Chunk 1) that handles transient network failures. Replace the single-line input with a textarea that supports multiline input with Shift+Enter.

File	Purpose	Key Changes from Current
src/hooks/useChat.ts	Chat state + SSE management	Add useReducer, streaming buffer (50ms flush), proper cleanup on unmount

File	Purpose	Key Changes from Current
src/components/chat/ChatPanel.tsx	Chat interface container	Debounced auto-scroll, textarea input, memoized message list
src/components/chat/ChatMessage.tsx	Single message renderer	React.memo, add rehype-sanitize to ReactMarkdown, fix XSS

Table 15. Chunk 6 Files

Acceptance Criteria:

- Chat streams tokens smoothly at 60fps even with 100+ message history
- Switching sidebar tabs and returning preserves chat history and SSE connection
- SSE connection automatically reconnects after transient network failures (with backoff)
- No XSS vectors through markdown rendering (verify with security audit)
- Multiline input works with Shift+Enter for newlines and Enter for send
- Suggestion chips trigger message send correctly without setTimeout hack
- Chat session persists across tab switches (stored in component state, not lost on unmount)

Bugs Resolved: C2 (tab switch), C3 (token re-render), C4 (XSS), C7 (SSE race condition), H12 (SSE reconnection)

4.8 Chunk 7: Sidebar Panels

Goal: Implement the remaining sidebar panels: YouTube Analytics (competitor videos + own channel stats), Skills viewer, and Knowledge Base viewer. These are simpler, data-display panels that fetch from REST endpoints and render static or semi-static content. The YouTube panel is the most complex of these, with lazy-loaded competitor data and a repurpose action.

Key Decisions: Only fetch competitor videos when the competitor tab is active (not always). Add loading="lazy" and onError handlers on all image thumbnails. Use mapApiError consistently across all error displays. Add rehype-sanitize to all markdown rendering (Skills, Knowledge Base). Implement proper loading skeletons instead of text-based loading indicators.

File	Purpose	Key Changes from Current
src/hooks/useYouTube.ts	YouTube analytics hooks	Remove dead apiFetcher code, fix type casting, conditional fetching
src/components/youtube/YouTubePanel.tsx	YouTube tab container	Conditional fetch based on active tab, add ARIA roles
src/components/youtube/CompetitorCard.tsx	Competitor video card	Add lazy loading, onError handler, aspect-ratio constraint
src/components/youtube/OwnStats.tsx	Channel statistics	Use mapApiError for error display, add refresh button
src/components/system/SkillViewer.tsx	Skill file viewer	Add rehype-sanitize, loading skeleton, retry on error
src/components/system/KnowledgeBase.tsx	Knowledge base viewer	Add rehype-sanitize, useCallback for loadContent

Table 16. Chunk 7 Files

Acceptance Criteria:

- Competitor videos are only fetched when the Competitors tab is active
- Broken thumbnail images show a fallback placeholder instead of a collapsed empty area
- All markdown content (Skills, Knowledge Base) is sanitized against XSS
- Error messages use the mapApiError system consistently across all panels
- Loading skeletons are shown while data is being fetched (not text "Loading..." messages)
- YouTube panel tabs have proper ARIA roles matching the Sidebar pattern

Bugs Resolved: H14 (image handling), M12 (dead code), M13 (XSS in KB), M14 (conditional fetch)

4.9 Chunk 8: Telemetry and System

Goal: Implement the telemetry and system information panels: QuotaPanel (live provider rate limits) and ModelRegistry (agent-to-model mapping). These are relatively simple display components, but the QuotaPanel requires real-time polling that should be conditional on panel visibility to avoid unnecessary API load.

Key Decisions: Make the QuotaPanel polling conditional on sidebar tab visibility using SWR isVisible option or a custom visibility hook. Use ProgressBar for RPM/TPM display if total limits are known from the API. ModelRegistry should ideally fetch from the backend rather than using hardcoded frontend data, but as a minimum, the current static display should be clearly labeled as "configuration-sourced" to set proper expectations. The MODEL_REGISTRY in types/index.ts must be updated to remove legacy "Groq" provider references and use the current FreeRouter providers (OpenRouter, Ollama Cloud).

File	Purpose	Key Changes from Current
src/hooks/useQuota.ts	Quota polling hook	Conditional polling based on panel visibility
src/components/telemetry/QuotaPanel.tsx	Rate limit display	Use mapApiError, add ProgressBar for RPM/TPM, add refresh indicator
src/components/telemetry/ModelRegistry.tsx	Model mapping display	Add note about data source (static). UPDATE MODEL_REGISTRY to remove Groq references and use current FreeRouter providers (OpenRouter, Ollama Cloud)

Table 17. Chunk 8 Files

Acceptance Criteria:

- Quota polling stops when the Quota tab is not visible
- Quota data is displayed with ProgressBar when total limits are available
- Error messages use mapApiError consistently
- Model registry clearly indicates whether data is static or dynamic
- MODEL_REGISTRY no longer references "Groq" as a provider; uses current FreeRouter providers

Bugs Resolved: M6 (conditional polling), M9 (MODEL_REGISTRY drift)

4.10 Chunk 9: Polish and Responsive

Goal: Apply cross-cutting polish across all chunks: responsive design for mobile and tablet viewports, accessibility audit and fixes, performance optimization, and final integration testing. This chunk does not add new

features but ensures the entire application works cohesively across different devices, screen sizes, and user scenarios.

Key Decisions: The application is primarily a desktop dashboard, so responsive design should focus on graceful degradation rather than full mobile optimization. The sidebar should collapse to a bottom sheet or overlay on mobile. The Kanban board should scroll horizontally on narrow viewports. All interactive elements must have proper ARIA labels and keyboard support. Performance profiling should identify and fix any remaining unnecessary re-renders. Clean up leftover Vite template assets (vite.svg, react.svg).

Area	Scope	Deliverables
Responsive Design	Sidebar collapse, horizontal scroll for Kanban	Works on 768px to 2560px viewports
Accessibility	ARIA labels, keyboard navigation, color contrast	WCAG 2.1 AA compliance audit
Performance	React.memo on all leaf components, profiling	60fps with 50+ cards, 100+ chat messages
Cleanup	Remove template assets, unused imports	Zero ESLint warnings
E2E Testing	Critical user flows	Tests for discover, drag, review, chat
Documentation	Component storybook or usage docs	README with setup and architecture

Table 18. Chunk 9 Scope

Acceptance Criteria:

- No ESLint warnings or TypeScript errors in the entire project
- Application renders correctly on viewports from 768px to 2560px
- All interactive elements are keyboard-accessible (Tab, Enter, Escape, Arrow keys)
- Color contrast meets WCAG 2.1 AA standards (4.5:1 for normal text, 3:1 for large text)
- No leftover Vite template files in the repository
- E2E tests pass for the 4 critical flows: topic discovery, card drag, script review, chat conversation

Bugs Resolved: M8 (duplicate animate-spin), remaining accessibility and performance issues

5. API Contract Reference

This section provides a quick-reference for all API endpoints available for frontend consumption. The backend has been significantly expanded since the original plan was written, now exposing 74 endpoints across 11 route groups. Each endpoint is listed with its HTTP method, path, the frontend component or hook that calls it, and the transport mechanism. Endpoints marked as "new" represent opportunities for expanded frontend functionality during recreation that were not previously integrated.

This contract should be used as the single source of truth when implementing the API client in Chunk 1 and when building components in subsequent chunks. The existing frontend currently consumes 16 of these endpoints; the remaining newly available endpoints represent significant opportunities for expanded functionality, including topic reservoir management, visual asset integration, memory/session inspection, dead letter queue monitoring, and a global event bus via SSE streaming.

Method	Endpoint	Frontend Consumer	Transport
POST	/api/kanban/topic-finder	Header.tsx (discover)	REST
GET	/api/kanban/tasks	useCards (SWR + Realtime)	Supabase
PATCH	/api/kanban/tasks/{id}	Board.tsx (drag move)	REST
DELETE	/api/kanban/tasks/{id}	Card.tsx (delete)	REST
POST	/api/kanban/tasks/{id}/soft-delete	Card.tsx (soft delete)	REST
POST	/api/kanban/tasks/{id}/undo-delete	Card.tsx (undo delete)	REST
PUT	/api/kanban/tasks/{id}/move	Board.tsx (drag move)	REST
POST	/api/kanban/tasks/{id}/save	Card.tsx (save topic)	REST
POST	/api/kanban/tasks/{id}/extend	Card.tsx (extend expiration)	REST
POST	/api/pipeline/discover	Header.tsx (discover)	REST
POST	/api/pipeline/produce/{id}	Card.tsx (produce)	REST
POST	/api/pipeline/langgraph/resume/{id}	ReviewPanel (approve/reject)	REST
GET	/api/pipeline/langgraph/state/{id}	usePipelineState (SWR 5s)	REST
POST	/api/chat/stream	useChat (SSE stream)	SSE
POST	/api/chat/message	ChatPanel (non-streaming)	REST
GET	/api/chat/history	ChatPanel (history)	REST
GET	/api/chat/models	ChatPanel (model list)	REST
GET	/api/providers/quota	useQuota (SWR 5s)	REST
GET	/api/analytics/competitors	useCompetitorVideos (SWR)	REST
GET	/api/analytics/channel	useOwnAnalytics (SWR)	REST
POST	/api/analytics/repurpose	CompetitorCard (repurpose)	REST
GET	/api/settings/skills	SkillViewer	REST
GET	/api/settings/knowledge-base	KnowledgeBase	REST
GET	/api/topics/reservoir	TopicReservoir (new)	REST
GET	/api/topics/{id}	TopicDetail (new)	REST
POST	/api/topics/{id}/approve	TopicApproval (new)	REST
POST	/api/topics/{id}/reject	TopicApproval (new)	REST
POST	/api/topics/custom	TopicSubmission (new)	REST
GET	/api/visual/manifests	VisualAssetPanel (new)	REST
GET	/api/visual/shaders	VisualAssetPanel (new)	REST
GET	/api/visual/templates	VisualAssetPanel (new)	REST
GET	/api/memory/sessions	MemoryPanel (new)	REST
GET	/api/memory/facts	MemoryPanel (new)	REST
GET	/api/dlq/stats	DeadLetterPanel (new)	REST
GET	/api/dlq/items	DeadLetterPanel (new)	REST

Method	Endpoint	Frontend Consumer	Transport
GET	/api/events	EventBus (SSE stream)	SSE

Table 19. Complete API Contract Reference (Updated)

Note: The backend now exposes 74 endpoints across 11 route groups. The frontend currently consumes 16 of these. The table above lists all endpoints relevant for frontend consumption, including newly available endpoints that were not previously integrated (marked as "new"). These represent opportunities for expanded frontend functionality during recreation, including topic reservoir management, visual asset browsing, memory session inspection, dead letter queue monitoring, and a global event bus via SSE streaming. Integrating these new endpoints should be prioritized based on user value and can be addressed in later development iterations beyond the initial 10-chunk recreation plan.

6. Testing Strategy

Each chunk should be validated through a combination of automated tests and manual verification. The testing strategy is designed to be lightweight but effective, focusing on the most critical paths and ensuring that each chunk can be independently verified before proceeding to the next. The goal is not 100% code coverage but rather confidence that the core functionality works correctly and that regressions are caught early.

Test Layer	Tool	Scope	When
Unit Tests	Vitest	API client, error mapper, card helpers, types/validation	Every chunk
Component Tests	React Testing Library	Card, Column, ReviewPanel, ChatMessage	Chunks 3-6
Integration Tests	MSW + RTL	Board DnD flow, Chat SSE flow, Review approval flow	Chunks 3-6
E2E Tests	Playwright	Critical user flows: discover, drag, review, chat	Chunk 9
Type Checking	tsc --noEmit	Full project type safety	Every chunk
Linting	ESLint	Code quality and consistency	Every chunk
Visual Regression	Playwright screenshots	Layout, Board, Sidebar, Card states	Chunk 9

Table 20. Testing Strategy by Layer

Chunk Gate Criteria: Before moving from one chunk to the next, the following must pass: (1) All unit tests for the current chunk pass, (2) TypeScript compiles with zero errors, (3) ESLint reports zero warnings, (4) Manual smoke test of the chunk functionality against the live backend, (5) No regressions in previously completed chunks. This gate process ensures that each chunk builds on a solid, verified foundation and that issues are caught early when they are cheapest to fix.

Mock Strategy: For unit and component tests, use MSW (Mock Service Worker) to intercept API calls and return deterministic responses. This allows testing against the exact API contract without needing a running backend. For integration tests, mock the Supabase Realtime WebSocket with a fake implementation that simulates card updates. For E2E tests, use the actual backend running in a test environment with a dedicated test database.

7. Risk Mitigation

Recreating a frontend from scratch carries inherent risks. This section identifies the primary risks and provides mitigation strategies for each. The key principle is that the existing backend remains the source of truth throughout the recreation, and each chunk is verified against it incrementally.

Risk	Likelihood	Impact	Mitigation
AI generates same bugs in new code	High	High	This plan documents every known bug with exact fixes; reference it as constraints when generating code for each chunk
Backend API contract changes during recreation	Medium	Medium	Lock the codebase-audit-finding-fixes branch before starting. Backend changes require updating the API contract table first
Supabase Realtime schema drift	Medium	Medium	Use Zod schemas at the Supabase boundary to validate incoming data. Fail visibly on schema mismatch
FreeRouter provider changes break model registry	Medium	Low	FreeRouter v3.1 is now a standalone LiteLLM proxy with 2 providers. MODEL_REGISTRY must use current provider names (OpenRouter, Ollama Cloud)
Context window limits for AI generation	High	Medium	Chunk-by-chunk approach limits context. Provide only the relevant plan section plus source files per chunk
DnD Kit API changes between versions	Low	Medium	Pin @dnd-kit versions. Use the TypeScript types to detect breaking changes during build
Tailwind CSS v4 class name changes	Low	Low	Use CSS-first configuration. Most v3 classes are preserved. Test visual output per chunk

Table 21. Risk Mitigation Matrix

Contingency Plan: If at any point during the recreation a chunk fails to validate against the existing backend, the approach is to stop, diagnose the mismatch, update the plan, and retry. The chunk-by-chunk approach ensures that failures are isolated and do not cascade. The existing frontend remains deployed and functional throughout the recreation process, so there is no user-facing downtime risk. Each completed chunk can be merged independently, allowing for incremental deployment as the rebuild progresses.

FreeRouter Considerations: With the removal of CrewAI and the migration to FreeRouter v3.1 as a standalone LiteLLM proxy, the frontend must not reference any legacy CrewAI or direct LiteLLM endpoints. All LLM interactions now flow through FreeRouter, which exposes 7 routes across 2 providers (OpenRouter, Ollama Cloud). The FreeRouter API keys must be stored in a separate freerouter/.env file, not in the main application .env, to maintain proper separation of concerns and security boundaries between the application layer and the LLM routing layer.